Introduction to Operating Systems

READ, RECALL, THEN PRACTISE

StudyNotes original PDF pages **6-12**. Notes begin on page 2 of this pack. Original content and annotations are preserved.

- Explain the OS role, resource abstractions and isolation.

- Trace a system call across the user/kernel boundary.

- Distinguish CPU/memory virtualization, concurrency and persistence.

SELECTED PRACTICE

MC-A: multiple-choice concepts / original PDF pages 1, 2

WR: short explanations / original PDF pages 1

Each practice section has a source page and question guide. Mixed-topic source pages may appear in more than one pack.

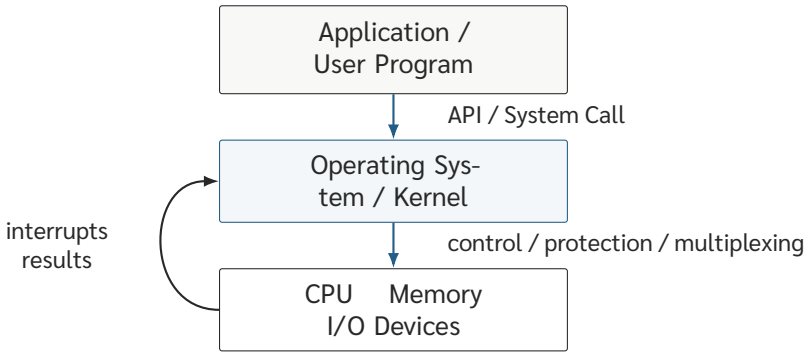
Lecture 1 — Introduction to Operating System

อ้างอิง: Lecture 1 หน้า 1–15 | บทบาทของ OS ระบบเรียกใช้บริการ การทำเสมือน และการจัดการทรัพยากร

1.1 Operating System อยู่ตรงไหนในระบบคอมพิวเตอร์

เนื้อหาจาก Lecture

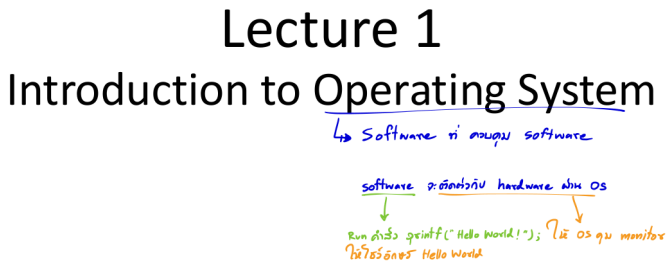
Operating System (OS) เป็นซอฟต์แวร์ที่ทำหน้าที่เป็นตัวกลางระหว่างโปรแกรมกับทรัพยากรฮาร์ดแวร์ โปรแกรมจึงไม่ต้องควบคุม CPU, memory, disk, display หรือ network device โดยตรงทุกครั้งที่ต้องการใช้งาน หมายเหตุในสไลด์หน้าแรกชี้ให้เห็นว่าโปรแกรมใช้ hardware ผ่าน OS ตัวอย่าง `printf("Hello World!")` แสดงให้เห็นว่าคำสั่งระดับโปรแกรมต้องอาศัยบริการของระบบก่อนข้อความจะแสดงผลบนอุปกรณ์



รายละเอียดสำคัญจากสไลด์

Lecture 1 หน้า 3 — ทบทวน Fetch / Decode / Execute สไลด์เชื่อมกับ Computer Architecture โดยอธิบายว่า processor ดึง instruction จาก memory (fetch) แปลความหมายของคำสั่ง (decode) แล้วจึง execute หรือเขียนผลลัพธ์กลับ จุดนี้ช่วยให้เห็นว่า OS ไม่ได้ execute instruction แทน CPU แต่จัด environment และควบคุมการทำงานรอบ instruction stream

ภาพประกอบ 1.1 Lecture 1 หน้า 1 — Software → OS → Hardware



จุดสังเกต

- ลูกศรเขียนมือชี้ว่า user software ไม่ได้ติดต่อ hardware โดยตรงอย่างไร้ข้อจำกัด
- ตัวอย่าง `printf()` สื่อเส้นทางจาก program ไปยัง service ด้านล่าง ไม่ได้หมายความว่า printf = system call ทุกกรณี
- ใช้ภาพนี้เป็นแบบจำลองตั้งต้นก่อนอ่านบทต่อไป

ที่มา: Lecture 1 หน้า 1 พร้อมหมายเหตุเขียนมือในสไลด์

OS ไม่ได้มีหน้าที่เพียงทำให้โปรแกรมเริ่มทำงาน แต่ยังต้องแบ่งทรัพยากรที่ใช้ร่วมกันให้โปรแกรมแต่ละตัวทำงานได้อย่างเป็นอิสระ พร้อมรักษาประสิทธิภาพ การควบคุม และการแยกขอบเขตของระบบ

1.2 System Call: ทำไมโปรแกรมทั่วไปเข้าถึง hardware โดยตรงไม่ได้

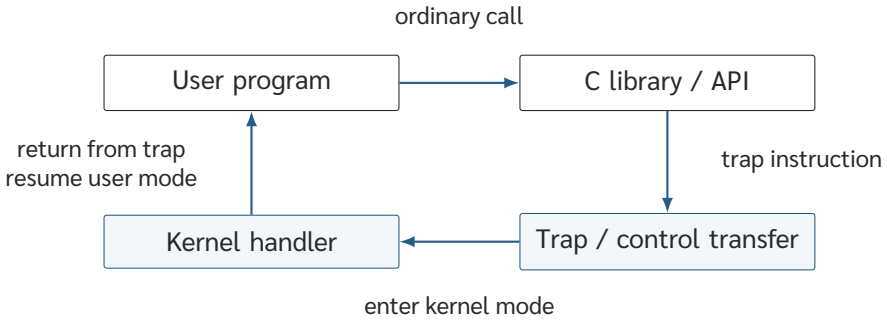
Lecture แนะนำ **User Mode**, **Kernel Mode** และ **System Call** ตั้งแต่ต้น เพื่ออธิบายว่า operation ที่มีสิทธิ์สูง เช่น การอ่านไฟล์หรือส่งข้อมูลผ่าน network ต้องผ่าน OS

	User Mode	Kernel Mode
ผู้ใช้งานหลัก	application / normal program	Operating System kernel
สิทธิ์	จำกัด; privileged operation ทำไม่ได้	เข้าถึง/ควบคุม hardware ได้
การขอ service	system call	kernel handler ดำเนินงาน
เป้าหมาย	isolation และความปลอดภัย	ทำงานที่ต้องใช้ privilege

1.2.1 System Call flow

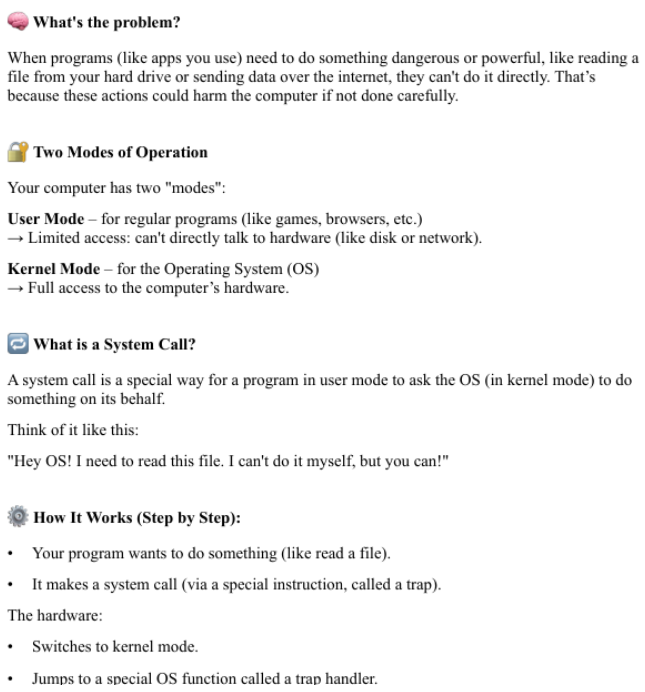
เมื่อ user program ต้องทำ operation ที่ถูกจำกัด ลำดับการทำงานเป็นดังนี้:

1. โปรแกรมเรียก API หรือ library function ที่เกี่ยวข้อง
2. library ใช้ special instruction หรือ convention เพื่อเรียก **trap**
3. hardware เปลี่ยนเป็น kernel mode และส่ง control ไปยัง **trap handler**
4. OS ทำ operation ที่ร้องขอด้วย privileged code
5. เมื่อทำงานเสร็จ OS ใช้ return mechanism เพื่อกลับไปยัง user program และลด privilege เป็น user mode



จุดที่มักสับสน

ภาพประกอบ 1.2 Lecture 1 หน้า 2 — User/Kernel Mode + System Call



What's the problem?

When programs (like apps you use) need to do something dangerous or powerful, like reading a file from your hard drive or sending data over the internet, they can't do it directly. That's because these actions could harm the computer if not done carefully.

Two Modes of Operation

Your computer has two "modes":

User Mode – for regular programs (like games, browsers, etc.)
→ Limited access: can't directly talk to hardware (like disk or network).

Kernel Mode – for the Operating System (OS)
→ Full access to the computer's hardware.

What is a System Call?

A system call is a special way for a program in user mode to ask the OS (in kernel mode) to do something on its behalf.

Think of it like this:

"Hey OS! I need to read this file. I can't do it myself, but you can!"

How It Works (Step by Step):

- Your program wants to do something (like read a file).
- It makes a system call (via a special instruction, called a trap).

The hardware:

- Switches to kernel mode.
- Jumps to a special OS function called a trap handler.

จุดสังเกต

- User Mode กับ Kernel Mode ต่างกันที่ระดับ privilege ไม่ใช่เพียงชนิดของโปรแกรม
- เมื่อต้องขอบริการ โปรแกรมเกิด trap เข้า kernel handler แล้ว return กลับสู่ user mode
- procedure call ปกติไม่ได้เปลี่ยน privilege boundary ด้วยตัวเอง

ที่มา: Lecture 1 หน้า 2 พร้อมคำอธิบายประกอบในสไลด์

Procedure Call ต่างจาก System Call procedure call เรียก function อื่นใน program/address space เดิมและไม่เปลี่ยน privilege level ด้วยตัวเอง ส่วน system call เปิดทางให้ user program ขอให้ kernel ทำ privileged operation

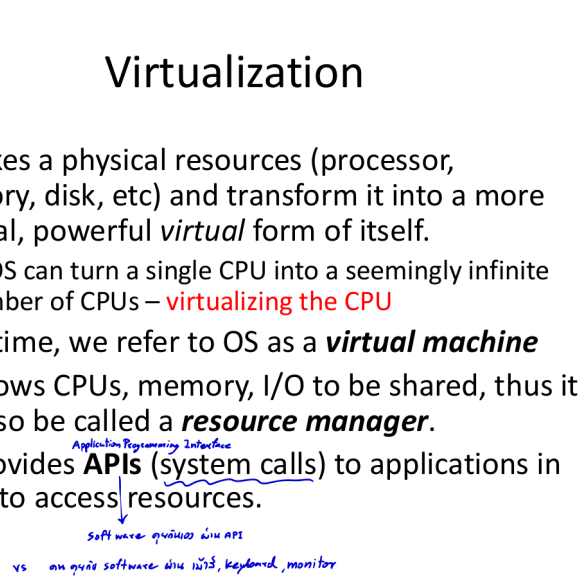
เชื่อมกับ Computer Architecture

เนื้อหา Computer Architecture เรื่อง I/O และ Processing Unit ช่วยอธิบายว่า OS ทำงานนี้ได้เพราะ hardware มีกลไกควบคุมสถานะของ processor ส่งต่อการทำงาน และจำกัด privileged operation ไม่ใช่เพราะซอฟต์แวร์สั่ง hardware ได้ทุกอย่างตามต้องการ ในบท Architecture ยังกล่าวถึง Privilege Exception และ return-from-interrupt ซึ่งเชื่อมโยงกับเส้นทาง trap/return ของ OS

1.3 Virtualization: ทำให้ physical resource ดูเหมือน resource ที่จัดการได้ง่ายขึ้น

Lecture นิยามภาพใหญ่ของวิชาผ่าน **Virtualization**

ภาพประกอบ 1.3 Lecture 1 หน้า 5 — Virtualization / Resource Manager



จุดสังเกต

- OS สร้าง abstraction จาก physical resources เพื่อให้โปรแกรมใช้งานได้สะดวกขึ้น
- OS ทำทั้ง **virtualization** และ **resource management**
- API และ System Call เป็น interface ที่ควบคุมได้ Virtualization จึงไม่ใช่เพียงการจำลองทรัพยากร

ที่มา: Lecture 1 หน้า 5.

OS รับ physical resource เช่น processor, memory, disk แล้วสร้าง abstraction ที่สะดวก ปลอดภัย และแชร์ได้

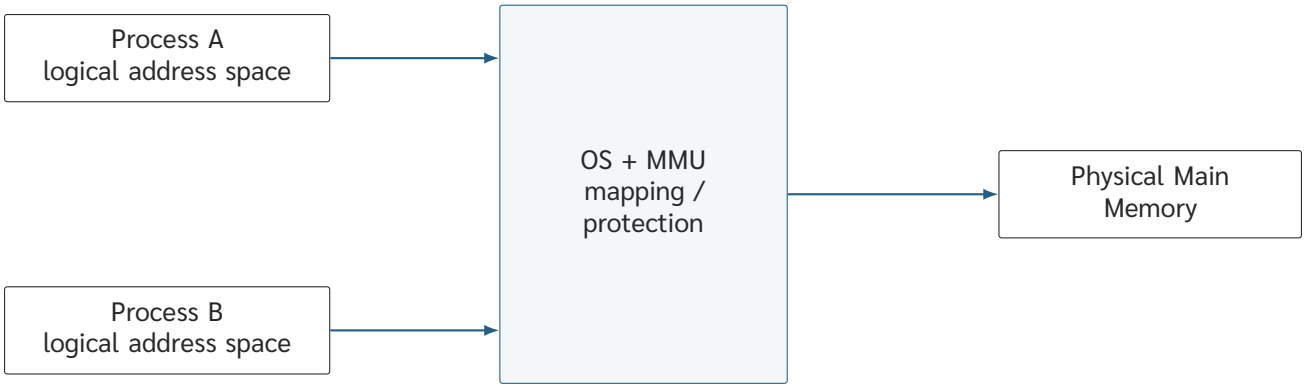
1.3.1 CPU Virtualization

หากมีหลายโปรแกรมต้องการทำงานบน processor core เดียว OS จะแบ่งเวลาและสลับการทำงานระหว่าง process เพื่อให้แต่ละ process ดูเสมือนว่ากำลังใช้ CPU อยู่ กลไกการสลับจะอธิบายใน Lecture 2 ส่วน policy ที่ใช้เลือก process ถัดไปอยู่ใน Lecture 3

> **อธิบายให้เห็นภาพ**
>
> แยกคำถามเรื่อง CPU virtualization ออกเป็นสองส่วน:
> - **Mechanism**: OS ดึง CPU กลับมาและสลับ process ได้อย่างไร?
> - **Policy**: เมื่อได้ CPU กลับมาแล้ว ควรเลือก process ใดให้ทำงานต่อ?
>
> การแยกสองคำนี้ตั้งแต่ Lecture 1 จะช่วยไม่ให้สับสนตอนเรียน Context Switch และ Scheduling

1.3.2 Memory Virtualization

ระหว่างที่โปรแกรมทำงาน ระบบจะอ่านและเขียนข้อมูลใน memory อยู่เสมอ OS จัดให้แต่ละ process มองเห็น address space ของตนเอง แม้ physical memory จะถูกใช้ร่วมกัน ใน Lecture 4–6 จะเริ่มจากความแตกต่างระหว่าง logical address กับ physical address แล้วต่อไปยัง Base/Bounds, Segmentation, Paging และ Virtual Memory



ภาพประกอบ 1.4 Process A/B ผ่าน MMU ไปยัง physical memory

ลูกศรสองเส้นแสดงว่า Process A และ B ส่ง logical address แยกกันเข้าสู่ MMU ระบบจะแปลง address และตรวจสิทธิ์ก่อนให้เข้าถึง physical memory ที่ใช้ร่วมกัน

1.4 OS as Resource Manager

นอกจากสร้าง abstraction แล้ว OS ยังเป็น **resource manager** โดยตัดสินใจว่า process ใดได้ใช้ CPU เมื่อใด จัด memory ให้ process อย่างไร จัดการ I/O ที่ต้องรอ และเก็บหรือเรียกคืนข้อมูล persistent ผ่าน API และ System Call ที่ OS ควบคุมได้

1.5 Concurrency: เมื่อหลายสิ่งดำเนินไปพร้อมกัน

Lecture ใช้ประโยค It's all about Concurrency เพื่อชี้ว่ากิจกรรมหลายอย่างเกิดซ้อนกันได้ ทั้ง process, I/O, interrupt, kernel work และ user work หากงานเหล่านี้เข้าถึง state ร่วมโดยไม่ประสานกัน ระบบอาจทำงานผิดพลาด
ใน Lecture 1 เรามอง Concurrency ในภาพรวมก่อน ประเด็นนี้ไม่ได้หมายถึงแค่การเปิดหลายโปรแกรมพร้อมกัน แต่รวมถึงลำดับที่งานสลับกันทำงานและการเข้าถึง shared state ซึ่ง OS ต้องจัดการให้ถูกต้อง

สไลด์หน้า 9 ยังชี้ว่า multithreaded program พบปัญหา concurrency ได้เช่นกัน แต่แนวทางจัดการต่างจาก concurrency ระดับ process เพราะ threads ภายใน process เดียวกันใช้ address space ร่วมกัน เรื่อง Thread API และ Locks/Semaphores เป็นหัวข้อที่แผนที่รายวิชาแยกไปเรียนภายหลัง

1.6 Persistence: ข้อมูลต้องอยู่รอดแม้ memory จะหาย

DRAM เป็น volatile storage เมื่อไฟดับ ข้อมูลใน main memory จะสูญหาย OS จึงจัดการข้อมูลบน non-volatile storage ผ่าน abstraction อย่าง **File System** เพื่อให้โปรแกรมสร้าง อ่าน เขียน และจัดระเบียบข้อมูลที่ต้องเก็บไว้ได้

Lecture ตั้งคำถามเพิ่มเติมว่าเหตุใด OS จึงไม่สร้าง private virtual disk แยกให้ทุก application ประเด็นที่ต้องจำคือ ส่วนนี้เน้นให้ OS จัดการ disk ร่วมกันผ่าน file system ไม่ได้ทำ virtualization แบบเดียวกับ CPU และ memory ในบทนี้

> **เชื่อมกับ Computer Architecture**
>
> มุมมองจาก Computer Architecture ช่วยแยกบทบาทและความเร็วของ memory hierarchy กับ I/O device โดย CPU และ main memory ทำงานเร็วกว่าการเข้าถึง disk มาก OS จึงต้องคำนึงถึง I/O latency เมื่อออกแบบ buffering, blocking, interrupt และ Virtual Memory/Page Fault

1.7 สิ่งที่ต้องคำนึงเมื่อออกแบบ OS

Lecture ระบุเป้าหมายการออกแบบหลายด้าน ซึ่งมักมี trade-off ต่อกัน:

ประเด็น	ความหมายในบริบท OS
Abstraction	สร้าง interface ให้ใช้งาน resource ที่ซับซ้อนได้ง่ายขึ้น
Performance	ลด overhead ที่เกิดจาก abstraction และการควบคุม
Isolation	ป้องกันไม่ให้ process หนึ่งอ่านหรือทำลาย state ของอีก process
Reliability	รักษาความถูกต้องแม้บาง component หรือ program มีปัญหา
Energy	ใช้ทรัพยากรและพลังงานอย่างเหมาะสม
Security	ควบคุมสิทธิ์และป้องกัน action ที่ไม่ได้รับอนุญาต
Mobility	แนวคิด OS ต้องรองรับบริบทอุปกรณ์หลากหลาย

1.8 Historical Motivation

สไลด์ท้าย Lecture สรุปพัฒนาการจาก **Early Mainframes** ไปสู่ **Multiprogramming on Minicomputer** และ OS บน PC สมัยใหม่ เมื่อคอมพิวเตอร์มีต้นทุนสูง การปล่อย CPU ว่างระหว่างรอ I/O ทำให้ใช้ทรัพยากรได้ไม่คุ้ม จึงเกิดแนวคิดให้หลายงานอยู่ในระบบและสลับใช้ทรัพยากร ส่งผลให้ต้องพัฒนา multiprogramming, protection และ OS ให้รองรับงานที่ซับซ้อนขึ้น

Lecture 1 หน้า 13–15 ให้ภาพพัฒนาการเพิ่มเติม: OS รุ่นแรกมีลักษณะเป็นชุด library สำหรับงาน I/O และมีคนควบคุมการรันทีละ program; เมื่อมีหลาย job อยู่ในระบบ การทำ **Multiprogramming** ช่วยใช้ CPU ระหว่างรอ I/O และนำมาสู่โจทย์ด้าน memory protection กับ interrupt. สไลด์เปรียบเทียบ DOS ซึ่งไม่มี memory protection, Mac OS รุ่นแรกที่จัด scheduling ได้ไม่ดี, Windows NT ที่นำแนวคิดจาก minicomputer OS มาใช้ และ Mac OS X ที่มี UNIX เป็นแกน พร้อมชี้การเปลี่ยนจากเครื่องที่แชร์กันทั้ง workgroup มาเป็นเครื่องส่วนบุคคล

สไลด์หน้า 4 เป็นแผนที่หัวข้อทั้งวิชา โดยยังมีเรื่อง Threads, Locks, Semaphores, Deadlock และ Mass Storage ที่เรียนต่อภายหลัง เอกสารนี้ยึดขอบเขตสอบกลางภาค Lecture 1–6 จึงระบุหัวข้อเหล่านั้นไว้เป็นแนวทาง แต่ไม่ขยายเป็นเนื้อหาบทเรียน

ใน course map ยังมี I/O System, File System และ Mini Projects ได้แก่ compile/build OS, ศึกษาและนำเสนอองค์ประกอบของ OS และแก้ไข kernel modules ด้วย หัวข้อเหล่านี้เป็นภาพรวมของรายวิชา ไม่ใช่เนื้อหาที่ Lecture 1–6 อธิบายเป็นบทเต็ม

> **จุดที่มักสับสน**
>
> ประวัติใน Lecture มีไว้ช่วยอธิบายว่าทำไมจึงต้องมี OS มากกว่าท่องปีหรือชื่อระบบ เมื่อเกิด multiprogramming ระบบก็ต้องมี isolation, scheduling และ memory management ที่รัดกุมขึ้น

1.9 Chapter Summary

> **สรุปท้ายบท**
>
> - OS เป็น software layer ที่ทำให้ application ใช้ hardware ผ่าน abstraction และ controlled interfaces
> - สามแกนใหญ่ของคอร์สคือ **Virtualization**, **Concurrency**, และ **Persistence**
> - CPU และ memory ถูก virtualize เพื่อให้หลาย process แชร์ physical resources ได้โดยมี isolation
> - System Call เป็น controlled entry จาก User Mode ไป Kernel Mode ผ่าน trap mechanism
> - OS เป็นทั้ง abstraction provider และ resource manager
> - design ต้องสมดุล convenience, performance, isolation, reliability, energy, security และ mobility

Key Mechanisms

User/Kernel Mode, Trap, Trap Handler, Return path, privileged execution.

Key Policies

Lecture 1 ยังไม่ลงรายละเอียด scheduling algorithm บทนี้วางกรอบไว้ว่า scheduler และ memory manager จะใช้ policy ต่าง ๆ บนกลไกที่ hardware รองรับ

ComArch Connections

Processor control state, privileged operations, I/O devices, interrupt/trap-style control transfer, physical memory.

Common Confusions

- OS ต่างจาก application library อย่างเดียว; kernel มี privilege และควบคุม resources
- System Call ต่างจาก ordinary procedure call
- Virtualization ต่างจาก simulation เสมอไป; ในคอร์สนี้คือการสร้าง abstraction/illusion เหนือ physical resource
- Concurrency ต่างจาก จำเป็นต้องมีหลาย CPU cores; interleaving บน CPU เดียวก็สร้าง concurrency problem ได้

Quick Revision Checklist

- ☐ อธิบาย Hardware และ OS และ Application ได้
- ☐ อธิบาย System Call ตั้งแต่ user code ผ่าน trap เข้าสู่ kernel จน return กลับได้
- ☐ แยก CPU virtualization กับ memory virtualization ได้
- ☐ อธิบายความหมายของ concurrency และ persistence ในภาพรวมได้
- ☐ บอกเหตุผลว่าทำไม User Mode/Kernel Mode ช่วย protection ได้

> **จุดเน้นก่อนสอบ**
>
> หลังจบ Lecture 1 ควรอธิบายความเชื่อมโยงได้ว่า **เมื่อโปรแกรมต้องการใช้ resource, OS จะให้บริการผ่าน abstraction และ API ส่วนงานที่ต้องใช้สิทธิ์สูงจะผ่าน kernel โดยอาศัย hardware mode และ control transfer เพื่อรักษา protection** แนวคิดนี้จะกลับมาอีกใน Process, Scheduling และ Virtual Memory

Practice: MC-A: multiple-choice concepts

ORIGINAL SOURCE

os midterm exam.pdf

Original PDF page(s): **1, 2**

Focus on questions 1-3 and 5. Other questions on these pages belong to Lecture 2.

ANSWER ANNOTATIONS

Handwritten, highlighted and pre-selected answers are part of the source and are not certified solutions. Hide them while practising. Recomputed examples and explicit assumptions are in CHECKED_DRILLS and the main study guide.

1. What is a trap in OS?

- A trap is a system call that provides OS level service
- A trap is a service that user can capture output from a process
- A trap is a system call that captures users in OS
- A trap is a way to capture bugs
- There is no trap in OS

2. What would happen if there is no user level and privileged level separation?

- Process cannot be kicked of processor
- OS cannot access disks or external devices
- Process may be able to access other process data
- OS can prevent process from directly access devices
- Process cannot create child process

3. Which of the following statements is true?

- Processes seem to occupy a CPU provided by modern OS
- an instruction in a program directly on disk
- A process is a program loaded into memory
- All of the statements are true
- None of the statements is true

4. What is not the component of a process?

Program

Data state

Process status

Control state

All of the choices are process components

5. What is the issue to consider to protect one application from others?

Performance

Isolation

Reliability

Mobility

All of the issues is for protection

6. What is the low-level hardware that is needed in non-preemptive system to switch process?

MMU

Timer interrupt

Cache

TLB

None of them

7. Which part of the process is in kernel space?

Text segment

Data segment

Practice: WR: short explanations

ORIGINAL SOURCE

WR / Written question collection (17 pages)

Full source filename is listed under this code in PRACTICE_INDEX.md.

Original PDF page(s): **1**

Focus on questions 2, 5 and 7. Use your own explanation rather than memorizing the annotations.

ANSWER ANNOTATIONS

Handwritten, highlighted and pre-selected answers are part of the source and are not certified solutions. Hide them while practising. Recomputed examples and explicit assumptions are in CHECKED_DRILLS and the main study guide.

1. What did Alan Turing was trying to solve when he built the first computer? (2 pts)

   Enigma Code

2. Will computer work without Operating System? Why? (2 pts)

   No, OS manage hardware resources and provide an environment for application to run.

3. What could be a result of non-preemptive process system? (2 pts)

   Process may not respond to high-priority task.

4. How can Operating System preempt a process from execution? (2 pts)

   Interupt process to switch task.

5. What is a trap in Operating System? What does it do? (4 pts)

   A trap is system call, provide OS level service.

6. What is the different between a process and a program? (2 pts)

   A process is an execution of instruction inside a program, but a program is a set of instruction

7. What is the dual-mode operation of Operating System? (2 pts)

   User mode & kernel mode

8. Draw the state diagram of a process with events that cause the transition. (5 pts)

